JAVA PARA INTERNET

CREDENCIADA JUNTO AO MEC PELA PORTARIA
N 3.455 DO DIA 19/11/2003

Sumário

NOSSA HISTÓRIA.....
INTRODUÇÃO 4
INTERNET COMO PLATAFORMA JAVA 5
JAVA EM APLICAÇÕES WEB 8
EXECUÇÃO DAS APLICAÇÕES 10
JAVA NO INTERNET EXPLORER..... 11
O PROTOCOLO HTTP 12
SERVLETS..... 13
JAVA SERVER PAGES 14
JAVA SERVER FACES..... 15
JAVA EXPRESSION LANGUAGE 17
JSP..... 18
STRUTS..... 19
HIBERNATE/JPA 21
JAVA WEB 22
JAVA EE..... 23
CONSIDERAÇÕES FINAIS 24
REFERÊNCIAS..... 25

NOSSA HISTÓRIA

A nossa história inicia com a realização do sonho de um grupo de empresários, em atender à crescente demanda de alunos para cursos de Graduação e Pós-Graduação. Com isso foi criado a nossa instituição, como entidade oferecendo serviços educacionais em nível superior.

A instituição tem por objetivo formar diplomados nas diferentes áreas de conhecimento, aptos para a inserção em setores profissionais e para a participação no desenvolvimento da sociedade brasileira, e colaborar na sua formação contínua. Além de promover a divulgação de conhecimentos culturais, científicos e técnicos que constituem patrimônio da humanidade e comunicar o saber através do ensino, de publicação ou outras normas de comunicação.

A nossa missão é oferecer qualidade em conhecimento e cultura de forma confiável e eficiente para que o aluno tenha oportunidade de construir uma base profissional e ética. Dessa forma, conquistando o espaço de uma das instituições modelo no país na oferta de cursos, primando sempre pela inovação tecnológica, excelência no atendimento e valor do serviço oferecido.

INTRODUÇÃO

O Java é uma linguagem de programação multiplataforma essencial na informática, inclusive para navegação web. Sem o software, os navegadores não conseguem, por exemplo, acessar sites de internet banking, além de outros recursos e aplicações.

O plugin Java utilizado nos navegadores, como o Internet Explorer, da Microsoft, é um dos componentes do Java Runtime Environment ou JRE. É um ecossistema de software para execução de conteúdos escritos em java, como applets gravados nesta linguagem e que são executados dentro do browser.

Sites de banco geralmente utilizam o Java para a autenticação do cliente e permissão do uso do Internet Banking. Quando este plugin não funciona, o usuário será impedido de realizar determinadas funções no navegador, como o acesso à sua conta bancária, por exemplo. Veja agora o que fazer caso o plugin Java não funcione no Internet Explorer.

Antes da implementação de qualquer código, saber determinados conceitos é extremamente importante. O primeiro conceito a ser trabalhado, pode parecer simples e sem necessidade de explicação, mas, por via das dúvidas, prefiro fazê-lo.

Plataforma

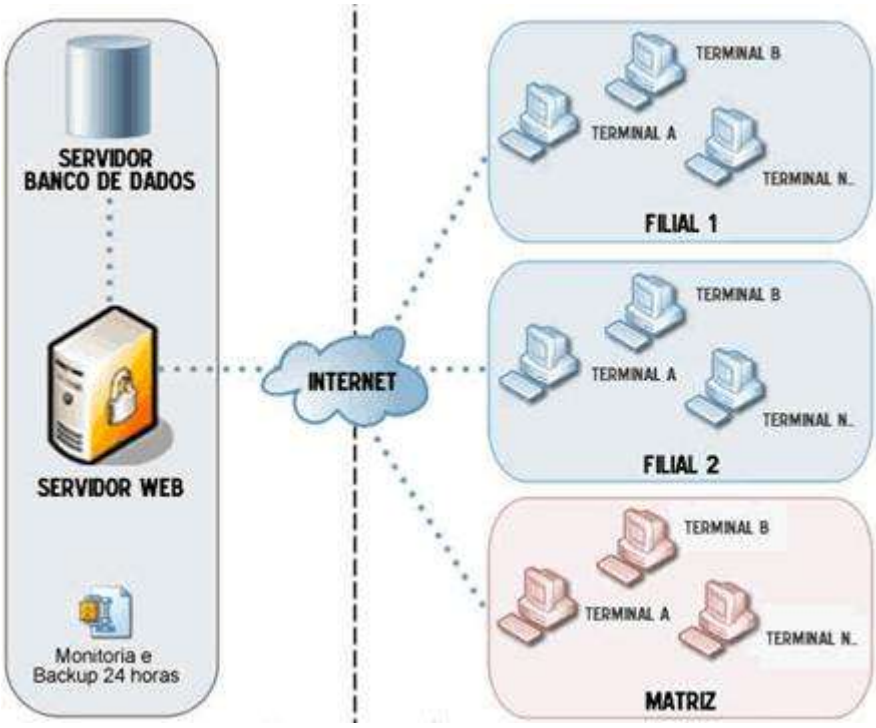
Pode-se dizer que plataforma é uma arquitetura específica, podendo esta arquitetura ser composta de hardware, software ou ambos. Uma plataforma determina a tecnologia empregada em infraestruturas específicas, buscando garantir facilidade de integração entre os diversos elementos existentes nesta infra-estrutura.

Pela definição exposta anteriormente para plataforma, pode-se definir que uma aplicação desenvolvida para a Internet, mais especificamente para a web, que é o foco desta apostila, tem como tecnologias empregadas para a infra-estrutura da mesma, de forma bem resumida: computadores e softwares servidores, equipamentos de redes de computadores, softwares específicos para aplicações web, como servidores de aplicações e/ou containeres de aplicações web, servidores de banco de dados e softwares navegadores.

Veja por esta simples definição que o desenvolvimento de uma aplicação web não é tarefa simples e nem para poucos profissionais da área, é preciso uma equipe. Mas existem muitas empresas que já oferecem toda a infra-estrutura pronta, restando para os desenvolvedores apenas a implementação da aplicação e disponibilização (distribuição/instalação) da mesma em servidores específicos.

Veja na Figura abaixo o esboço de uma plataforma web, com servidores e serviços específicos (lado esquerdo da figura) e a indicação que as aplicações hos-

pedadas nestes equipamentos são acessadas através de um conjunto de equipamentos, simulando três redes independentes, possivelmente em lugares geograficamente diferentes (lado direito). Veja que, ligando os equipamentos que necessitam acessar as aplicações e os equipamentos que hospedam as aplicações existe uma rede, a Internet.



(Fonte: http://www.virto.com.br/diferenciais.aspx)

Arquitetura

Pode-se conceituar arquitetura como um conjunto de componentes (quer seja hardware ou software), suas propriedades externas e seus relacionamentos com outros componentes.

Com a definição de arquitetura exposta, deve-se afirmar que uma arquitetura voltada para uma aplicação web (que é uma plataforma específica) possui componentes que precisam conhecer uns aos outros, saber o que precisam para integrarem entre si e então garantir o perfeito funcionamento de uma aplicação web. Por exemplo, um servidor que hospede uma aplicação web pode ter nele,

servidores de aplicação, servidores de bancos de dados e servidores de autenticação. Cada um deles precisando conhecer as propriedades externas dos outros, para que possam se comunicar sempre que necessário pela aplicação.

O desenvolvimento de aplicações desktop com Java SE (Java Standard Edition) e suas bibliotecas AWT, Swing e Applets se tornou bastante difundido com o aumento no uso da linguagem Java. Isto se deve, dentre outros motivos, à portabilidade das aplicações desenvolvidas, uma das grandes inovações do Java.

Neste cenário de aplicações desktop, imagine uma aplicação que foi criada para o controle de acesso de uma empresa, sendo esta instalada em 200 máquinas. Quando essa aplicação sofrer alguma alteração negocial e precisar ser atualizada, todas as versões instaladas nas máquinas da empresa precisarão passar por uma atualização.

Assim, as dificuldades de manutenção, atualização e distribuição de aplicações desktop, aliadas à necessidade de instalação de uma versão do aplicativo nas máquinas do usuário, motivou a busca por uma melhor forma de distribuir aplicações que fossem acessadas por vários usuários e pudessem estar disponíveis em qualquer lugar.

Seguindo o exemplo anterior, se nossa aplicação estivesse em um ponto centralizado, onde seu acesso fosse distribuído, onde as alterações fossem efetuadas em apenas uma única versão da aplicação, e não houvesse a necessidade de instalação desta aplicação nas máquinas do usuário, teríamos uma melhor disponibilidade de nossa aplicação, assim como um melhor controle sobre suas alterações.

Este ponto centralizado veio com a utilização da plataforma web no desenvolvimento de aplicações.

No desenvolvimento web, o cliente usa um navegador (browser) para acessar a aplicação, necessitando apenas a instalação do navegador em sua máquina.

Como a utilização dos navegadores foi difundida com a popularização da Internet, a plataforma web foi beneficiada neste ponto.

Mas o que é essencial para criarmos aplicações web utilizando a linguagem Java? Segundo o Netbeans.org, uma aplicação Java Web gera páginas web interativas, que podem conter vários tipos de linguagens de marcação (HTML, XHTML, XML, entre outras) e conteúdo dinâmico, sendo estas páginas compostas por componentes web como: JavaServer Pages (JSP), servlets e JavaBeans.

Esses componentes são responsáveis pelo armazenamento temporário dos dados que serão transitados a cada requisição/resposta em uma página na web.
Com base em tudo o que falamos até aqui, neste artigo iremos lembrar a trajetória do Java na plataforma web, iniciando com uma breve análise sobre o protocolo HTTP, e falando sobre a importância dos servlets e do JSP no desenvolvimento de aplicações. Depois mostraremos o panorama atual dos principais frameworks de mercado voltados ao desenvolvimento web.

EXECUÇÃO DAS APLICAÇÕES

Existem dois ou mais pontos logicamente (na maioria das vezes fisicamente) separados na arquitetura para aplicações a serem executados na plataforma Internet. Que são os serviços oferecidos, normalmente a aplicação web fica “hospedada” neste lado. E os interessados em utilizar os serviços oferecidos.

A separação física é mais clássica. Quando você acessa os serviços oferecidos pelo seu banco, você sabe realmente onde estão os serviços requisitados? Onde estão os seus dados pessoais e de transações que são trazidos para sua máquina? A separação lógica ocorre mais no momento de implementação, quando em uma única máquina têm-se os servidores, serviços oferecidos e desta mesma máquina, estes recursos são acessados.

Independente de como a separação dos recursos e dos interessados neles exista, é preciso saber que as aplicações web, acessadas via *browser* são acessadas e atendidas através de requisições e respostas (*request* e *response*).

O browser estando em um computador qualquer em qualquer parte do mundo e, precisa de um serviço que está disponibilizado em um outro equipamento, em qualquer parte do mundo. Para utilizar este serviço, ele requisita o mesmo, através do browser, que envia esta requisição através do protocolo HTTP (HyperText Transfer Protocol). O servidor, por sua vez, recebe a requisição, juntamente com dados que vêm junto ao cabeçalho HTTP, como dados informados pelo usuário em um formulário em uma página web. Neste momento, o servidor identifica o serviço e define quem é responsável por ele.

O responsável, então, atende a requisição e gera uma resposta para o solicitante (o cliente). Um detalhe importante é saber que quem atende uma requisição normalmente é uma aplicação, implementada em qualquer linguagem, mas a resposta, gerada para o cliente, é sempre HTML, pois o browser, sabe ler (por enquanto) apenas HTML.

JAVA NO INTERNET EXPLORER

O Internet Explorer possui um recurso de segurança chamado “bloqueio de controle ActiveX”. Ele serve para manter atualizado os controles do ActiveX. E o Java é um deles. Se o Java estiver desatualizado, este recurso de segurança irá bloquear sua execução.

A resolução deste problema é simples: basta atualizar o Java para a sua versão mais recente. O próprio Internet Explorer vai mostrar um pop-up sugerindo a atualização, exibindo um link. Caso não queira usar este método, basta ir no site da Oracle e baixar a versão mais recente.

Essa situação é válida para o Windows 7 com SP1 e Internet Explorer versões 8 a 11, e o Windows 8 com a versão nativa do Internet Explorer.

O PROTOCOLO HTTP

A comunicação entre navegadores e uma aplicação web é realizada por meio de requisições e respostas definidas pelo protocolo HTTP. Sendo assim, os desenvolvedores de aplicações web devem estar preparados para trabalhar com o protocolo HTTP, conhecendo este a fundo.

O protocolo HTTP permite a clientes e servidores interagir e trocar informações de uma maneira simples e confiável, com a utilização de URIs (Uniform Resource Indentifier) para identificar dados na Internet. Estes identificadores, aliados às localizações dos documentos no servidor, são chamados de URLs (Uniform Resource Locators).

URLs fazem referência a arquivos, diretórios ou objetos situados em um servidor web e que serão acessados a partir de um navegador ou diretamente a partir de outras aplicações.

SERVLETS

Para armazenarmos nossos recursos em um servidor web, utilizamos um contêiner web. Ele será responsável pelo envio e recebimento de requisições HTTP, além de garantir que nossa aplicação seja acessada de forma simultânea por vários usuários, e que todo conteúdo do sistema seja gerado de forma dinâmica. No mercado existem alguns contêineres web famosos, como o Tomcat e o Jetty. Também existem os servidores de aplicação, que implementam toda ou parte da especificação Java EE (Java Enterprise Edition), como TomEE, JBoss AS, GlassFish, dentre outros.

O contêiner web é responsável por gerenciar o ciclo de vida dos nossos recursos, mapear uma URL para um recurso particular e garantir que o requisitante da URL possua os direitos de acesso corretos a este recurso. Estes recursos Java que possuem o ciclo de vida controlado pelo contêiner web são denominados servlets.

Os servlets foram definidos a partir de uma especificação Java que está atualmente na versão 3.1 e se encontra detalhada na JSR-340. O objetivo dos Servlets era padronizar a interface dos recursos disponibilizados pelo contêiner web. Os servlets utilizam o protocolo HTTP para realizar o processamento dos dados de um formulário HTML e recuperar informações que são transmitidas a partir desse protocolo, fornecendo conteúdo dinâmico, como dados vindos de uma consulta no banco de dados.

Até a especificação 2.5, para a criação de um Servlet o desenvolvedor necessitava criar a sua classe herdando da classe `HttpServlet` e, além disto, configurá-lo corretamente no arquivo `web.xml`. Com a versão 3.0, a declaração do Servlet passou a ser feita também através do uso de anotações.

A tecnologia JSP, acrônimo de JavaServer Pages, é uma linguagem de script baseada em uma especificação Java, a JSR-245, que está atualmente na versão 2.1 e é utilizada para criação de páginas com conteúdo dinâmico.

O uso de páginas JSP fez com que o conteúdo HTML, antes contido em servlets, ficasse em um arquivo separado, com a extensão `.jsp`.

Além do conteúdo HTML é possível a utilização de scriptlets nas páginas JSP. Os scriptlets são códigos escritos dentro dos delimitadores `<%` e `%>` que servem para embutir nessas páginas trechos de código Java puro

O uso de scriptlets nas páginas JSP fez com que os códigos que antes ficavam apenas nas classes Java fossem colocados em arquivos JSP, incluindo a realização dos imports das classes Java utilizadas. Assim, para fazer uma listagem, por exemplo, o trecho de código Java se mistura ao código HTML.

O JSF, acrônimo de JavaServer Faces, nasceu de uma especificação Java, a JSR-127, em 2001, sendo lançada sua primeira release em 2004. Baseado na arquitetura MVC, surgiu da necessidade de se ter um framework que fosse orientado à interface, como o Swing para aplicações desktop, porém sendo robusto para lidar com aplicações web. Atualmente, ele se encontra na segunda versão, o JSF 2.2, que foi concebido sobre a especificação JSR 344.

Por ser uma especificação, várias empresas se propõem a implementar versões para o JSF. Dentre as implementações mais famosas, temos:

Mojarra: implementação de referência do JSF, e a mais utilizada;

MyFaces: implementação open source da Apache;

ADF Faces: implementação open source da Oracle.

Cada uma destas implementações nos oferece apenas componentes básicos, que na grande maioria dos sistemas corporativos não é suficiente, sendo necessária àutilização de conjuntos de componentes complexos e ricos. Entre algumas das soluções Java que fornecem bons conjuntos de componentes, podemos destacar: o RichFaces, PrimeFaces e Tomahawk.

Ser uma especificação Java facilitou na rápida aceitação do JSF no mercado, principalmente para as grandes empresas, que precisam ter ao menos alguma garantia de continuidade da tecnologia. Esta aceitação fez com que o mercado passasse a utilizar componentes em suas aplicações para a criação da interface com o usuário.

O controle fornecido pelo JSF sobre o que acontece nos componentes é viabilizado graças ao armazenamento das representações de cada elemento como um objeto em uma árvore de componentes. Cada elemento desta árvore guarda o estado atual dos componentes da tela e, a cada evento disparado, é feita uma requisição para o JSF, que se encarrega de atualizar as informações nesta árvore no início da requisição.

Além disso, como o JSF tem um modelo de componentes extensível, é possível a criação de novos componentes que se encaixam nas necessidades de sua aplicação. Esta extensibilidade foi aproveitada na construção dos frameworks RichFaces, PrimeFaces, entre outros, que criaram componentes úteis e práticos, como componentes de layout, temas, componentes drag-and-drop e gráficos.
Com o JSF já consolidado, sua nova versão, o JSF 2.2, veio para corrigir alguns problemas da primeira e acrescentar importantes funcionalidades, como o suporte nativo a Ajax, o que aumentou o dinamismo no desenvolvimento, e principalmente o uso de anotações, o que diminuiu as configurações com XML da primeira versão. Destaque também para a integração do JSF 2.2 com o restante da Java EE 7.

JAVA EXPRESSION LANGUAGE

Visando diminuir a quantidade de código Java presente nas páginas JSP, surgiram as Expression Language (EL). Estas facilitam o acesso aos dados armazenados em componentes JavaBeans, sendo possível, com a sua utilização, acessar o valor destes componentes em páginas JSP.

As ELs permitem a criação de expressões aritméticas e lógicas. Dentro de uma expressão EL você pode usar inteiros, números de ponto flutuante, strings, constantes de verdadeiro e falso para valores booleanos, e nulos.

Para acessar atributos de um objeto JavaBean usamos a seguinte sintaxe: ${bean.propriedade}, onde bean representa o JavaBean a ser acessado e propriedade o atributo. Ao utilizar esta sintaxe, o método get da propriedade do JavaBean é invocado.

Vejamos outro exemplo: suponha a existência do bean Usuario com a propriedade Endereco, que também é um Bean. Se desejarmos exibir a propriedade Bairro do Bean Endereco, devemos utilizar a seguinte sintaxe: ${usuario.endereco.bairro}.

Podemos também acessar coleções como Arrays, Lists e Maps diretamente em nossas JSPs. Para isto, considerando um ArrayList como variável de escopo, este seria acessado com a seguinte sintaxe: ${arrayList[posiçãoDaEntrada]}, onde com um ArrayList para representar as Cidades, usaríamos a sintaxe: ${cidades[1]}, ou no caso de uma HashMap para representar os Estados, teríamos ${estados[“DF”]}.

Como vimos, podemos utilizar servlets para trabalharmos sobre o protocolo HTTP, interceptando suas requisições, exibindo conteúdo HTML de nossa escolha e enviando-o como resposta ao navegador do usuário, assim como podemos utilizar páginas JSP para trabalharmos com conteúdo dinâmico, acessando nossas classes Java para exibirmos listagens vindas diretamente do banco de dados, por exemplo.

Mas a pergunta que não quer calar é: Seria melhor utilizarmos servlets ou JSP em nossas aplicações? Para responder essa pergunta, vamos analisar os pontos de cada uma das tecnologias. Com servlets, temos o controle do protocolo HTTP. Assim, controlamos nossas requisições e respostas, além de podermos implementar alguma regra pertinente ao negócio e realizarmos acessos ao banco de dados da aplicação, sendo possível escrevermos o código de saída que será apresentado ao usuário, tudo em uma mesma classe.

Com JSP, temos a construção de páginas nas quais podemos utilizar trechos de código Java, ou tags da biblioteca JSTL, para apresentar conteúdo dinâmico ao usuário, acessando assim classes Java, que podem trazer listagens vindas do banco de dados, por exemplo, mas ficamos desamparados na implementação das regras de negócio, visto que o foco do JSP, por ser uma linguagem de script, é a apresentação de conteúdo.

Percebemos então que os servlets possuem o foco na camada de negócio da aplicação, enquanto as JSPs possuem foco na camada de apresentação. Deste modo, temos que nossa melhor resposta está na utilização de ambos, servlets e JSP, empregando o melhor de cada tecnologia.

STRUTS

Quando os servlets foram criados, logo foi percebido que eles eram uma ferramenta rápida e poderosa para o desenvolvimento de aplicações web, já que eram extensíveis, e logo seu uso tornou-se difundido em aplicações JSP.

Ao longo da utilização dos servlets, percebeu-se que as implementações que empregavam o padrão MVC eram muito próximas umas das outras, e muito código era genérico, ou seja, o mesmo código era utilizado por diversas aplicações, independente das funcionalidades destas aplicações.

Devido a essas semelhanças surgiram alguns frameworks visando reaproveitar esses códigos genéricos, criando assim implementações reaproveitáveis seguindo o padrão MVC. Dentre eles, o de maior destaque foi o Apache Struts, um dos primeiros frameworks para a camada de Controller na plataforma Java.

O projeto Struts foi criado em maio de 2000, por Craig R. McClanahan, para fornecer um framework MVC padrão para o desenvolvimento de aplicações web em Java, e em julho de 2001, a primeira versão, o Struts 1.0, foi lançado oficialmente.

O Struts contava com a separação provida pela arquitetura MVC, além de validações de dados dos formulários, internacionalização e integração com outras tecnologias adotadas pelo mercado, o que viabilizava maior produtividade na implementação.

À época o Struts foi considerado um grande avanço para a comunidade Java, tanto que sua adoção nos projetos, principalmente no mercado de grandes empresas, foi alta, sendo sua primeira versão utilizada até hoje.

A arquitetura do framework Struts se baseia no MVC, sendo que representando a camada Controller temos um servlet chamado de ActionServlet. Este atua como

o controlador que recepciona todas as requisições advindas do navegador, encaminhando estas requisições de acordo com as configurações XML efetuadas no arquivo struts-config.xml.

Temos também a classe Java denominada Action, que é baseada em ações a serem executadas junto aos objetos de negócio.

Com a alta utilização, as deficiências do Struts foram aparecendo. Dentre elas podemos destacar as excessivas configurações em XML que precisavam ser realizadas, fazendo com que simples alterações gerassem grandes impactos na aplicação, além do grande uso de herança das classes do framework Struts, o que gerava um alto acoplamento.

Dessa forma, na busca por alternativas a estes problemas, os desenvolvedores do Struts implementaram uma nova versão, baseada no projeto WebWork, um framework concorrente que possuía ideias que iam de encontro com as alternativas de melhoria do framework Struts.

Assim, em 2005 surgiu a segunda versão do framework, o Struts 2, que é continuado até hoje. No entanto, esta versão não obteve o mesmo sucesso da primeira, e não foi adotado em larga escala pelas empresas, já que existiam mais opções no mercado, além de já ter sido lançada uma especificação Java baseada no uso de componentes web, o JSF.

HIBERNATE/JPA

As aplicações criadas com Java possuem a orientação a objetos como o seu principal paradigma. No entanto, quando necessitamos interagir com bancos de dados, precisamos também trabalhar com o paradigma relacional. Neste contexto, visando facilitar a integração Objeto-Relacional, as ferramentas ORM (Object-Relational Mapping) ganharam espaço entre os desenvolvedores.

O Hibernate é uma ferramenta ORM que surgiu em 2001 por iniciativa de Gavin King e logo se tornou amplamente utilizado. Ele conta com uma grande diversidade de recursos para o mapeamento objeto relacional. Atualmente, como líder de mercado, o Hibernate tem alguns dos seus recursos utilizados como base para a especificação JPA

Uma aplicação Java Web gera páginas Web interativas, que contêm vários tipos de linguagem de marcação (HTML, XML, etc.) e conteúdo dinâmico. Normalmente é composto por componentes Web, como JavaServer Pages (JSP), servlets e JavaBeans para modificar e armazenar dados temporariamente, interagir com bancos de dados e Web services e processar o conteúdo como resposta às solicitações do cliente.

Como a maioria das tarefas envolvidas no desenvolvimento de aplicações Web, pode ser repetitiva ou exigir um excedente de código padrão, os frameworks da Web podem ser aplicados para aliviar a sobrecarga associada às atividades comuns. Muitos frameworks, como JavaServer Faces, fornecem, por exemplo, bibliotecas para páginas de modelo e gerenciamento de sessão, e geralmente fomentam a reutilização do código.

JAVA EE

O Java EE (Enterprise Edition) é uma plataforma amplamente usada que contém um conjunto de tecnologias coordenadas que reduz significativamente o custo e a complexidade do desenvolvimento, implantação e gerenciamento de aplicações de várias camadas centradas no servidor. O Java EE é construído sobre a plataforma Java SE e oferece um conjunto de APIs (interfaces de programação de aplicações) para desenvolvimento e execução de aplicações portáteis, robustas, escaláveis, confiáveis e seguras no lado do servidor.

Alguns dos componentes fundamentais do Java EE são:

Enterprise JavaBeans (EJB): uma arquitetura gerenciada de componente do lado do servidor utilizada para encapsular a lógica de negócios de uma aplicação. A tecnologia EJB permite o desenvolvimento rápido e simplificado de aplicações distribuídas, transacionais, seguras e portáteis baseadas na tecnologia Java.

Java Persistence API (JPA): um framework que permite aos desenvolvedores gerenciar os dados utilizando o mapeamento relacional de objetos (ORM) em aplicações construídas na plataforma Java.

CONSIDERAÇÕES FINAIS

Como vimos, as especificações Java voltadas para web, inicialmente com servlets e JSP, tiveram uma grande aceitação no mercado, mas a demora em encontrar soluções para abstrair as dificuldades de se trabalhar com o protocolo HTTP fez com que frameworks de terceiros ganhassem espaço.

Neste contexto, vale destacar inicialmente a fundação Apache, com seu framework Struts, visto que mesmo com sua primeira versão, que conta com conceitos hoje defasados, descontinuada, ainda persiste em boa parte das aplicações de grandes empresas do mercado, sendo possível um programador ter que prestar manutenção em aplicações que utilizem este framework com frequência.

Atualmente, o destaque do desenvolvimento web está na grande utilização das especificações Java, que com o JSF como carro chefe, vem ganhando cada vez mais espaço no mercado com tecnologias como JPA, EJB, CDI, e demais especificações da plataforma Java EE (agora na versão 7), dando a segurança de continuidade que as empresas precisam para criar suas aplicações.

REFERÊNCIAS

https://www.techtudo.com.br/dicas-e-tutoriais/noticia/2015/07/como-ativar-o-java-no-chrome-firefox-e-internet-explorer.html

https://www.techtudo.com.br/dicas-e-tutoriais/noticia/2016/03/problemas-com-plugin-do-java-no-internet-explorer-veja-dicas.html

https://www.devmedia.com.br/introducao-ao-desenvolvimento-de-aplicacoes-web/29798

http://www.linhadecodigo.com.br/artigo/2002/web-uma-introducao-ao-desenvolvimento-para-a-internet-com-java-parte-01-preparando-o-ambiente-para-o-desenvolvimento.aspx

https://netbeans.apache.org/kb/docs/java-ee_pt_BR.html#:~:text=Uma%20aplica%C3%A7%C3%A3o%20Java%20Web%20gera,e%20conte%C3%BAdo%20din%C3%A2mico.&text=Muitos%20frameworks%2C%20como%20JavaServer%20Faces,fomentam%20a%20reutiliza%C3%A7%C3%A3o%20do%20c%C3%B3digo.